

QAIRT Orchestrator System Design: Engineering Specification

****Project Core:**** Multi-Process Qwen 3.5 9B Q4_0 Deployment on Android
****Master Runtime:**** QAIRT (Qualcomm AI Runtime) & llama.cpp (Hexagon HVX)
****Master Folder:**** `QAIRT\_Orchestrator\_System\_Design`
****Documentation Date:**** Jul 08, 2026

Engineering Roadmap & System Architecture

1. High-Level Design & Process Isolation

To maintain peak system stability and zero impact on gaming frame rates, the architecture utilizes a decoupled daemon strategy.

- * ****Decoupled Daemon Strategy:**** AI inference is locked inside isolated native background processes (e.g., `:qairt\_engine`). This ensures memory management is invisible to the main rendering loop, protecting high-priority CPU/GPU lanes.

- * ****Fault Isolation:**** If an engine daemon exhausts its token window or hits a tensor parsing bug, only that specific process dies. The Master Orchestrator Service re-initializes the backend in milliseconds without crashing the UI or the active game.

- * ****Privileged System Layout:**** Utilizing system-level privileges allows the daemon to monitor the active gaming window and drive real-time AI assistance.

2. Core Orchestration & Hardware Virtualization

The orchestrator manages multiple backends and bridges the gap between hardware constraints and large model requirements.

- * ****Split-Model QAIRT Architecture:**** To bypass the 3.5GB hardware memory mapping limit on Qualcomm NPUs, the Qwen 3.5 9B model (requiring ~5.5GB-6GB) is split across virtual devices HTP0 and HTP1.

- * ****Dual-Payload Asset Pattern:**** The orchestrator maps user-facing `.gguf` files to corresponding pre-compiled `.bin` context binaries when the QAIRT backend is active, while supporting raw `.gguf` for llama.cpp fallbacks.

- * ****Native Core (FFI):**** A Rust or C++ core handles high-speed pipelines and tokenizers, allowing Kotlin/Java to remain a shallow UI shell.

3. High-Speed Data Transport (IPC)

Zero-copy mechanisms are implemented to eliminate CPU data-copying taxes and performance bottlenecks.

* **Shared Memory (memfd_create):** Uses secure, anonymous RAM cache blocks for game state mapping and tensor outputs.

* **UNIX Domain Sockets (UDS):** Bridges file descriptors between engines using POSIX ancillary payloads (`SCM\_RIGHTS`) within Linux's abstract namespace, bypassing disk I/O entirely.

* **Serialization Protocol:** Implements FlatBuffers or lightweight Protobuf for low-overhead message streaming between daemons.

4. Lifecycle Management & CI/CD

* **Sticky Foreground Service:** The orchestrator registers as a Foreground Service with `FOREGROUND\_APP\_ADJ` priority to protect heavy model processes from the Android Low Memory Killer (LMK).

* **Automated Multi-Binary Matrix:** A GitHub Actions pipeline compiles separate C++, Rust, and Kotlin artifacts, packaging them into a single APK.

Critical Data Hygiene: Duplicate & Redundant Files

To maintain a single source of truth, the following redundant files should be archived:

* **Google System Alignment:** "Number six" (File 8), "Number six" (File 9), and "6" (File 10) are identical technical guides on `AndroidManifest` and `ASharedMemory`.

* **Distributed Architecture:** "4" (File 6) and "5" (File 5) provide duplicate blueprints for the Rust/Kotlin daemon management.

* **Orchestrator Blueprint:** Both copies of "Second reply" (File 7 and File 11) detail the same Hexagon-Optimized pattern.

* **System Layout:** "Number eight" (File 0) and "Number n" (File 2) contain identical code for `memfd\_create` and NPU binding.

Would you like me to generate the specific C++ ring-buffer implementation for your game variable updates or the GitHub Actions YAML for the multi-binary build?